

Guía de laboratorio de la semana 7

Thymeleaf expresiones atributos y plantillas

Curso Marcos de Desarrollo Web

En este laboratorio transformaremos el catálogo estático de la semana 6 en vistas HTML generadas en el servidor. El controlador enviará datos mediante Model, Thymeleaf los presentará con expresiones seguras y los fragmentos evitarán repetir la cabecera y el pie.

Línea del laboratorio Java 25 LTS, Spring Boot 4.1.1, Thymeleaf 3.1.5.RELEASE y Bootstrap 5.3.8.

Alcance de esta práctica

Conservaremos la API REST de la semana 6 para comparar JSON con HTML renderizado. No añadiremos todavía base de datos, formularios de escritura, autenticación ni autorización. Estas responsabilidades pertenecen a semanas posteriores.

Producto a desarrollar

- Proyecto TechLab actualizado sin perder la API REST ni los archivos del APF1.
- Catálogo dinámico, búsqueda por GET y página de detalle servidos por Thymeleaf.
- Fragmentos, mensajes, estilos y página de error 4xx organizados por responsabilidad.
- Pruebas MVC, matriz de resultados, evidencias visuales y README actualizado.

1. Logro y organización del laboratorio

Al finalizar, el estudiante construye vistas dinámicas con Spring MVC y Thymeleaf, distingue el modelo de presentación del dominio, usa expresiones y atributos del motor de plantillas, reutiliza fragmentos y verifica rutas, modelos, estados HTTP y HTML resultante.

Criterios observables

- El controlador de vistas usa `@Controller`, inyección por constructor y nombres de vista fijos.
- La lista emplea `th:each`, `th:if`, `th:unless`, `th:text`, `th:value` y `th:href` correctamente.
- El detalle usa `th:object` y expresiones de selección sin duplicar la consulta del servicio.
- La cabecera y el pie provienen de fragmentos; los textos reutilizables están externalizados.
- Las entradas tienen límite, los datos se muestran con escape y los errores conservan su estado HTTP.
- Las pruebas automatizadas y manuales demuestran rutas, modelo, renderizado y accesibilidad básica.

Distribución sugerida

Bloque	Trabajo verificable
1	Modelo mental, preparación y comparación REST frente a vistas
2	Dependencias, controlador de vistas, mensajes y fragmentos
3	Plantillas de lista, detalle, errores y hoja de estilos
4	Pruebas automatizadas, navegador y depuración
5	Reto, evidencias y reflexión

2. De una respuesta JSON a una vista del servidor

La API de la semana 6 devuelve datos que JavaScript interpreta. En esta semana, el mismo servicio alimenta un Model y Thymeleaf produce el HTML antes de enviarlo al navegador. Ambas rutas pueden coexistir porque atienden consumidores diferentes.

Aspecto	API REST	Vista Thymeleaf
Controlador	<code>@RestController</code>	<code>@Controller</code>
Retorno	Objeto que se serializa	Nombre lógico de plantilla
Respuesta	JSON	HTML procesado
Presentación	Cliente JavaScript	Servidor con Thymeleaf
Ruta TechLab	<code>/api/v1/cursos</code>	<code>/cursos</code>

Flujo de la vista dinámica

1. El navegador solicita `GET /cursos?q=Spring`.
2. `DispatcherServlet` selecciona `CourseViewController.listar`.
3. El controlador normaliza `q` y solicita los datos a `CourseService`.
4. El controlador agrega cursos y `q` al Model y devuelve `cursos/lista`.

5. El ViewResolver encuentra templates/cursos/lista.html.
6. Thymeleaf evalúa expresiones, reemplaza fragmentos y genera HTML.
7. El navegador recibe HTML; no recibe el objeto Model ni ejecuta Thymeleaf.

Idea clave. Model es un contenedor de atributos válido durante la solicitud. No es una entidad JPA, una tabla ni una sesión. La plantilla debe presentar datos; no debe abrir conexiones, consultar repositorios ni decidir reglas de negocio.

3. Expresiones y atributos esenciales de Thymeleaf

Sintaxis	Propósito	Ejemplo en TechLab
<code>\${...}</code>	Variable del contexto	<code>\${curso.titulo}</code>
<code>*{...}</code>	Propiedad del objeto seleccionado	<code>*{horas}</code>
<code>#{...}</code>	Mensaje externalizado	<code>#{accion.buscar}</code>
<code>@{...}</code>	URL consciente del contexto	<code>@{/cursos/{id}(id=\${curso.id})}</code>
<code>~{...}</code>	Referencia a fragmento	<code>~{fragments/layout :: pie}</code>

Los atributos `th:text` y `th:value` sustituyen texto y valores; `th:each` itera; `th:if` y `th:unless` controlan presencia; `th:href` construye enlaces; `th:object` establece un objeto de selección; y `th:replace` sustituye una etiqueta por un fragmento reutilizable.

Seguridad de salida. `th:text` escapa HTML de forma predeterminada. No cambie a `th:utext` con datos del usuario, porque interpretaría marcado y aumentaría el riesgo de XSS. Tampoco construya el nombre de una plantilla a partir de `q` o de otro parámetro de la solicitud.

Plantilla natural. El contenido estático dentro de cada etiqueta sirve como ejemplo cuando el archivo se abre sin el servidor. Al renderizar, los atributos `th:*` sustituyen ese contenido sin impedir que el HTML siga siendo legible como prototipo.

4. Preparar el proyecto de la semana 6

1. Abra la carpeta TechLab que contiene `pom.xml` y los scripts `mvnw`.
2. Cree un commit o una copia recuperable antes de modificar dependencias y rutas.
3. Ejecute `./mvnw test` y confirme que las pruebas de la semana 6 terminan correctamente.
4. Inicie la aplicación y confirme `/actuator/health` y `/api/v1/cursos`.
5. Conserve `CourseController` y su API REST. Solo reemplazará la navegación del portal.

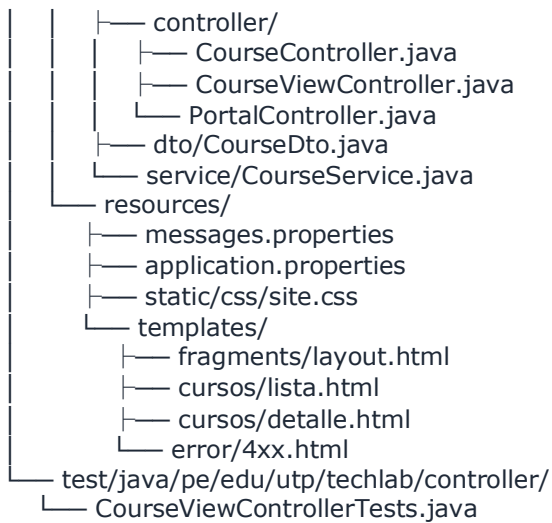
En Windows PowerShell use `.\mvnw.cmd` en lugar de `./mvnw`. El proyecto debe ejecutarse con Java 25 tanto en la terminal como en el IDE.

Estructura objetivo

```

techlab-web/
├── pom.xml
├── mvnw
├── mvnw.cmd
├── .mvn/wrapper/
└── src/
    ├── main/
    │   ├── java/pe/edu/utp/techlab/
    │   └── TechLabWebApplication.java
    └── test/
        ├── java/pe/edu/utp/techlab/
        └── TechLabWebApplicationTest.java

```



Spring Boot busca las plantillas en `src/main/resources/templates` y los recursos públicos en `src/main/resources/static`. No coloque plantillas Thymeleaf bajo `static`: allí se servirían sin procesar.

Reglas de continuidad

- No duplique la lista de cursos dentro de una plantilla ni en un nuevo script JavaScript.
- No elimine `CourseController`: la API REST debe seguir disponible para otros clientes.
- No cambie el paquete raíz; Spring Boot detecta componentes desde `pe.edu.utp.techlab`.
- No suba `target` ni datos sensibles al repositorio; documente los comandos en `README`.

5. Actualizar las dependencias

Reemplace pom.xml por el contenido completo siguiente o incorpore únicamente las dependencias nuevas si su archivo conserva metadatos institucionales. No declare una versión independiente para Thymeleaf: Spring Boot administra un conjunto compatible.

pom.xml parte 1 de 2

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>

  <groupId>pe.edu.utp</groupId>
  <artifactId>techlab-web</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>techlab-web</name>
  <description>Proyecto acumulativo del curso</description>

  <properties>
    <java.version>25</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
```

pom.xml parte 2 de 2 continuación inmediata

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

spring-boot-starter-thymeleaf agrega el motor y su integración con Spring MVC. Los starters webmvc-test y thymeleaf-test aportan el entorno específico para probar rutas, modelos y vistas. DevTools se mantiene opcional y no se empaqueta como dependencia obligatoria de consumidores.

src/main/java/pe/edu/utp/techlab/TechLabWebApplication.java completo

```
package pe.edu.utp.techlab;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TechLabWebApplication {

    public static void main(String[] args) {
        SpringApplication.run(TechLabWebApplication.class, args);
    }
}
```

src/main/resources/application.properties completo

```
spring.application.name=techlab-web
server.port=8080
management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=never
```

No establezca spring.thymeleaf.cache=false en la configuración de producción. Durante el desarrollo, DevTools aplica valores adecuados; si el IDE no actualiza una plantilla, reinicie la aplicación antes de alterar propiedades globales.

6. Conservar el DTO y el servicio

La vista no necesita otra copia de los datos. Reutilizar el mismo CourseService evita que el catálogo HTML y la API REST diverjan. Si sus archivos coinciden con la semana 6, compárelos y manténgalos; se incluyen completos para que la práctica sea verificable.

src/main/java/pe/edu/utp/techlab/dto/CourseDto.java completo

```
package pe.edu.utp.techlab.dto;

public record CourseDto(long id, String titulo, int horas) {
}
```

src/main/java/pe/edu/utp/techlab/service/CourseService.java completo

```
package pe.edu.utp.techlab.service;

import java.util.List;
import java.util.Locale;
import java.util.Optional;

import org.springframework.stereotype.Service;

import pe.edu.utp.techlab.dto.CourseDto;

@Service
public class CourseService {

    private final List<CourseDto> cursos = List.of(
        new CourseDto(1L, "HTML y CSS", 12),
        new CourseDto(2L, "Bootstrap", 16),
        new CourseDto(3L, "Spring Boot", 20)
    );

    public List<CourseDto> buscar(String texto) {
        String criterio = Optional.ofNullable(texto)
            .orElse("")
            .strip()
            .toLowerCase(Locale.ROOT);

        if (criterio.isBlank()) {
            return cursos;
        }

        return cursos.stream()
            .filter(curso -> curso.titulo()
                .toLowerCase(Locale.ROOT)
                .contains(criterio))
            .toList();
    }

    public Optional<CourseDto> buscarPorId(long id) {
        return cursos.stream()
            .filter(curso -> curso.id() == id)
            .findFirst();
    }
}
```

El record define un DTO inmutable. List.of y Stream.toList producen colecciones no modificables. Locale.ROOT hace que la normalización no dependa del idioma del sistema. En la semana 9 el origen en memoria se reemplazará por persistencia sin trasladar la consulta a la plantilla.

7. Crear el controlador de vistas

Cree un controlador separado del REST. La anotación `@Controller` permite que el String devuelto se interprete como nombre de vista. La inyección por constructor hace explícita la dependencia y facilita las pruebas.

src/main/java/pe/edu/utp/techlab/controller/CourseViewController.java completo

```
package pe.edu.utp.techlab.controller;

import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.server.ResponseStatusException;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.service.CourseService;

@Controller
@RequestMapping("/cursos")
public class CourseViewController {

    private static final int MAX_QUERY_LENGTH = 60;

    private final CourseService service;

    public CourseViewController(CourseService service) {
        this.service = service;
    }

    @GetMapping
    public String listar(
        @RequestParam(defaultValue = "") String q,
        Model model) {
        String consulta = normalizar(q);
        model.addAttribute("cursos", service.buscar(consulta));
        model.addAttribute("q", consulta);
        return "cursos/lista";
    }

    @GetMapping("/{id}")
    public String detalle(@PathVariable long id, Model model) {
        CourseDto curso = service.buscarPorId(id)
            .orElseThrow(() -> new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Curso no encontrado"));
        model.addAttribute("curso", curso);
        return "cursos/detalle";
    }

    private String normalizar(String q) {
        String consulta = q.strip();
        if (consulta.length() > MAX_QUERY_LENGTH) {
            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST,
                "La búsqueda admite hasta 60 caracteres");
        }
        return consulta;
    }
}
```



```
}
}
```

listar agrega dos atributos: cursos alimenta la iteración y q conserva el criterio en el campo. detalle devuelve 404 cuando el servicio no encuentra el ID. Ambos métodos retornan nombres constantes; nunca se utiliza entrada del usuario como nombre de plantilla.

Actualizar la navegación

Reemplace PortalController para que la raíz y /portal conduzcan a la nueva vista. redirect: produce una respuesta 302 con Location; el navegador realiza después GET /cursos.

src/main/java/pe/edu/utp/techlab/controller/PortalController.java completo

```
package pe.edu.utp.techlab.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class PortalController {

    @GetMapping("/")
    public String inicio() {
        return "redirect:/cursos";
    }

    @GetMapping("/portal")
    public String portal() {
        return "redirect:/cursos";
    }
}
```

Conserve CourseController.java de la semana 6 sin cambios. Compruebe que /api/v1/cursos continúa entregando JSON antes de seguir.

8. Externalizar mensajes y crear fragmentos

messages.properties centraliza textos que se repiten o podrían localizarse. La expresión #{clave} los resuelve según el Locale. No almacene allí secretos, credenciales ni datos que cambian durante la ejecución.

src/main/resources/messages.properties completo

```
app.nombre=TechLab Web
nav.cursos=Cursos
nav.api=API JSON
curso.lista.titulo=Catálogo de cursos
curso.detalle.titulo=Detalle del curso
curso.buscar=Buscar cursos
curso.buscar.placeholder=Ejemplo: Spring
curso.resultados={0} curso(s) encontrado(s)
curso.horas=Horas
curso.ver=Ver detalle
accion.buscar=Buscar
accion.limpiar=Limpiar
accion.volver=Volver al catálogo
```

Ahora cree un documento de fragmentos. head recibe un parámetro; cabecera y pie no necesitan argumentos. Los enlaces usan @{} para respetar el contexto de despliegue. Bootstrap se carga con versión fija, integridad SRI y crossorigin.

src/main/resources/templates/fragments/layout.html parte 1 de 2

```
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head th:fragment="head(titulo)">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title th:text="${titulo} + ' | TechLab Web'">TechLab Web</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
    rel="stylesheet"
    integrity="sha384-sRI4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
    crossorigin="anonymous">
  <link rel="stylesheet"
    href="../static/css/site.css"
    th:href="@{/css/site.css}">
</head>
<body>
<header th:fragment="cabecera" class="border-bottom bg-white">
  <nav class="container d-flex flex-wrap align-items-center gap-3 py-3"
    aria-label="Navegación principal">
    <a class="navbar-brand fw-semibold me-auto"
      href="/cursos"
      th:href="@{/cursos}"
      th:text="#{app.nombre}">TechLab Web</a>
```

src/main/resources/templates/fragments/layout.html parte 2 de 2 continuación inmediata

```
<a class="link-body-emphasis text-decoration-none"
  href="/cursos"
  th:href="@{/cursos}"
  th:text="#{nav.cursos}">Cursos</a>
<a class="link-body-emphasis text-decoration-none"
  href="/api/v1/cursos"
  th:href="@{/api/v1/cursos}"
  th:text="#{nav.api}">API JSON</a>
</nav>
</header>

<footer th:fragment="pie" class="border-top bg-white mt-auto">
  <div class="container py-3 small text-body-secondary">
    TechLab Web · Marcos de Desarrollo Web · 2026
  </div>
</footer>
</body>
</html>
```

th:replace sustituye por completo el elemento receptor. Usar fragmentos evita mantener tres copias del menú. Si cambia una ruta o el nombre de la aplicación, la modificación se realiza en un solo archivo.

9. Construir la lista dinámica

Cree lista.html en tres partes consecutivas. Es un único archivo: no repita doctype, html ni body entre partes. El formulario usa GET porque consultar no cambia estado y permite compartir la URL.

src/main/resources/templates/cursos/lista.html parte 1 de 3

```
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head({curso.lista.titulo})}">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Catálogo de cursos</title>
  <link rel="stylesheet" href="../../static/css/site.css">
</head>
<body class="d-flex flex-column min-vh-100">
<a class="visually-hidden-focusable position-absolute p-2 bg-white"
  href="#contenido">Saltar al contenido</a>

<header th:replace="~{fragments/layout :: cabecera}">
  <nav><a href="/cursos">TechLab Web</a></nav>
</header>

<main id="contenido" class="container py-4">
  <div class="row g-4 align-items-end mb-4">
    <div class="col-lg-7">
      <p class="text-primary fw-semibold mb-2">Semana 7</p>
      <h1 class="display-6 fw-bold"
        th:text="#{curso.lista.titulo}">Catálogo de cursos</h1>
      <p class="lead mb-0">
        Vista dinámica generada en el servidor con Spring MVC y Thymeleaf.
      </p>
    </div>
  </div>
</div>
```

src/main/resources/templates/cursos/lista.html parte 2 de 3 continuación inmediata

```
<form class="row g-3 align-items-end mb-4"
method="get"
th:action="@{/cursos}">
<div class="col-md-8">
  <label class="form-label" for="q"
    th:text="#{curso.buscar}">Buscar cursos</label>
  <input class="form-control" id="q" name="q" type="search"
    maxlength="60"
    th:value="{q}"
    th:placeholder="#{curso.buscar.placeholder}"
    placeholder="Ejemplo: Spring">
</div>
<div class="col-md-4 d-flex gap-2">
  <button class="btn btn-primary" type="submit"
    th:text="#{accion.buscar}">Buscar</button>
  <a class="btn btn-outline-secondary"
    th:if="{!#strings.isEmpty(q)}"
    th:href="@{/cursos}"
    th:text="#{accion.limpiar}">Limpiar</a>
</div>
</form>

<p class="text-body-secondary"
  th:text="#{curso.resultados({#lists.size(cursos)})}">
  3 curso(s) encontrado(s)
</p>

<div class="alert alert-info" role="status"
  th:if="{#lists.isEmpty(cursos)}">
  No se encontraron cursos. Pruebe con otro término.
</div>
```

src/main/resources/templates/cursos/lista.html parte 3 de 3 continuación inmediata

```
<div class="row g-4" th:unless="{#lists.isEmpty(cursos)}">
  <div class="col-md-6 col-xl-4" th:each="curso : {cursos}">
    <article class="card course-card h-100 shadow-sm">
      <div class="card-body d-flex flex-column">
        <p class="text-body-secondary mb-2">
          Código <span th:text="{curso.id}">1</span>
        </p>
        <h2 class="h4 card-title" th:text="{curso.titulo}">
          HTML y CSS
        </h2>
        <p class="card-text">
          <span th:text="{curso.horas}">12</span>
          <span th:text="{#curso.horas}">Horas</span>
        </p>
        <a class="btn btn-outline-primary mt-auto align-self-start"
          href="detalle.html"
          th:href="@{/cursos/{id}(id={curso.id})}"
          th:text="{#curso.ver}">Ver detalle</a>
      </div>
    </article>
  </div>
</div>
</main>

<footer th:replace="~{fragments/layout :: pie}">
  <p>TechLab Web · 2026</p>
</footer>
</body>
</html>
```

Cómo leer la plantilla

- th:value conserva q después de enviar el formulario; maxlength ayuda al cliente, pero el servidor también valida.
- #lists.size y #lists.isEmpty consultan la colección sin introducir lógica de acceso a datos.
- th:each crea una tarjeta por CourseDto y limita la variable curso al elemento repetido.
- th:if muestra el estado vacío; th:unless evita renderizar una cuadrícula sin elementos.
- @{/cursos/{id}(id={curso.id})} construye una URL con variable de ruta correctamente codificada.
- Los textos estáticos de respaldo permiten entender el prototipo sin ejecutar el servidor.

Comprobación inmediata. Ejecute la aplicación, abra /cursos y use Ver detalle. En Ver código fuente del navegador no deben aparecer atributos th:* ni expresiones \${...}; el servidor debe haberlos procesado.

10. Crear el detalle y la vista de errores

La página de detalle demuestra expresiones de selección. `th:object` establece `curso` como objeto actual y `*{...}` accede a sus componentes. No consulte el servicio desde la plantilla.

src/main/resources/templates/cursos/detalle.html completo

```
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head(# {curso.detalle.titulo})}">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Detalle del curso</title>
  <link rel="stylesheet" href="../../static/css/site.css">
</head>
<body class="d-flex flex-column min-vh-100">
<a class="visually-hidden-focusable position-absolute p-2 bg-white"
  href="#contenido">Saltar al contenido</a>

<header th:replace="~{fragments/layout :: cabecera}">
  <nav><a href="/cursos">TechLab Web</a></nav>
</header>

<main id="contenido" class="container py-5">
  <section class="mx-auto detail-panel" th:object="${curso}">
    <p class="text-primary fw-semibold"
      th:text="# {curso.detalle.titulo}">Detalle del curso</p>
    <h1 class="display-6 fw-bold" th:text="* {titulo}">Bootstrap</h1>
    <dl class="row mt-4">
      <dt class="col-sm-4">Código</dt>
      <dd class="col-sm-8" th:text="* {id}">2</dd>
      <dt class="col-sm-4" th:text="# {curso.horas}">Horas</dt>
      <dd class="col-sm-8" th:text="* {horas} + ' horas'">16 horas</dd>
    </dl>
    <a class="btn btn-primary mt-3"
      href="lista.html"
      th:href="@{/cursos}"
      th:text="# {accion.volver}">Volver al catálogo</a>
  </section>
</main>

<footer th:replace="~{fragments/layout :: pie}">
  <p>TechLab Web · 2026</p>
</footer>
</body>
</html>
```

Spring Boot permite plantillas de error por código o familia. El archivo `4xx.html` atiende errores del cliente, como una búsqueda demasiado larga o un curso inexistente, sin convertirlos en 200.

src/main/resources/templates/error/4xx.html completo

```
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Solicitud no válida')}">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Solicitud no válida</title>
  <link rel="stylesheet" href="../../static/css/site.css">
</head>
<body class="d-flex flex-column min-vh-100">
<a class="visually-hidden-focusable position-absolute p-2 bg-white"
  href="#contenido">Saltar al contenido</a>
<header th:replace="~{fragments/layout :: cabecera}">
  <nav><a href="/cursos">TechLab Web</a></nav>
</header>

<main id="contenido" class="container py-5">
  <section class="mx-auto detail-panel" aria-labelledby="error-title">
    <p class="display-4 fw-bold text-primary mb-2"
      th:text="${status}">4xx</p>
    <h1 id="error-title" class="h2">No se pudo completar la solicitud</h1>
    <p>Revise la dirección o los datos enviados e inténtelo nuevamente.</p>
    <p class="small text-body-secondary">
      Ruta: <code th:text="${path}">/cursos/999</code>
    </p>
    <a class="btn btn-primary" th:href="@{/cursos}" href="/cursos">
      Volver al catálogo
    </a>
  </section>
</main>

<footer th:replace="~{fragments/layout :: pie}">
  <p>TechLab Web · 2026</p>
</footer>
</body>
</html>
```

La vista usa status y path, atributos del manejo estándar de errores. No muestre trazas, nombres de clases ni mensajes internos a usuarios finales. El estado HTTP sigue siendo la fuente de verdad.

11. Añadir una hoja de estilos accesible

Bootstrap resuelve la estructura responsiva; site.css conserva la identidad visual y mejora el foco. El movimiento de las tarjetas solo se activa cuando el sistema no solicita reducción de movimiento.

src/main/resources/static/css/site.css completo

```
:root {
  color-scheme: light;
}

body {
  background: #f6f8fb;
  color: #1f2937;
}

.visually-hidden-focusable:focus {
  z-index: 1080;
  outline: 3px solid #0d6efd;
  outline-offset: 2px;
}
```



```

a:focus-visible,
button:focus-visible,
input:focus-visible {
  outline: 3px solid #0b5ed7;
  outline-offset: 3px;
}

.course-card {
  border: 1px solid #dbe3ec;
}

.detail-panel {
  max-width: 44rem;
  padding: clamp(1.25rem, 4vw, 3rem);
  background: #ffffff;
  border: 1px solid #dbe3ec;
  border-radius: 1rem;
}

@media (prefers-reduced-motion: no-preference) {
  .course-card {
    transition: transform 0.2s ease, box-shadow 0.2s ease;
  }

  .course-card:hover {
    transform: translateY(-0.15rem);
  }
}

```

No agregue estilos dentro de las plantillas. Mantener CSS externo facilita caché, reutilización y revisión. Pruebe el foco con Tab y confirme que el enlace Saltar al contenido aparece al recibir foco.

12. Ejecutar y rastrear una solicitud

Terminal en Linux macOS o Git Bash

```

./mvnw clean test
./mvnw spring-boot:run

```

Terminal en Windows PowerShell

```

.\mvnw.cmd clean test
.\mvnw.cmd spring-boot:run

```

1. Abra <http://localhost:8080/cursos> y compruebe tres tarjetas.
2. Busque Spring y observe q=Spring en la barra de direcciones.
3. Abra </cursos/2> y confirme Bootstrap, código 2 y 16 horas.
4. Abra </api/v1/cursos> y confirme que la respuesta sigue siendo JSON.
5. Abra </cursos/999> y confirme el estado 404 en la pestaña Network.
6. Revise la consola: no debe haber errores ni recursos CSS con 404.

Para distinguir servidor y cliente, use Ver código fuente. El HTML recibido ya contiene los cursos. En la semana 6, catalogo.js recibía JSON y creaba la interfaz; en esta ruta no hay fetch.

13. Probar controlador modelo y renderizado

MockMvc ejercita la capa MVC sin abrir un puerto. @WebMvcTest carga los controladores y la infraestructura web; @Import incorpora el servicio real con datos deterministas. El test se presenta en dos partes para copiar en un único archivo.

src/test/java/pe/edu/utp/techlab/controller/CourseViewControllerTests.java parte 1 de 2

```
package pe.edu.utp.techlab.controller;

import static org.hamcrest.Matchers.containsString;
import static org.hamcrest.Matchers.hasSize;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.model;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.view;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.web.servlet.MockMvc;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.service.CourseService;

@WebMvcTest({CourseViewController.class, PortalController.class})
@Import(CourseService.class)
class CourseViewControllerTests {

    @Autowired
    private MockMvc mvc;

    @Test
    void listadoRenderizaModeloYHtml() throws Exception {
        mvc.perform(get("/cursos"))
            .andExpect(status().isOk())
            .andExpect(view().name("cursos/lista"))
            .andExpect(model().attribute("q", ""))
            .andExpect(model().attribute("cursos", hasSize(3)))
            .andExpect(content().string(
                containsString("Catálogo de cursos")));
    }
}
```

src/test/java/pe/edu/utp/techlab/controller/CourseViewControllerTests.java parte 2 de 2 continuación inmediata

```
@Test
void filtroConservaConsultaYReduceResultados() throws Exception {
    mvc.perform(get("/cursos").param("q", "Spring"))
        .andExpect(status().isOk())
        .andExpect(model().attribute("q", "Spring"))
        .andExpect(model().attribute("cursos", hasSize(1)))
        .andExpect(content().string(containsString("Spring Boot")));
}

@Test
void detalleRenderizaElCursoSolicitado() throws Exception {
    mvc.perform(get("/cursos/2"))
        .andExpect(status().isOk())
        .andExpect(view().name("cursos/detalle"))
        .andExpect(model().attribute(
            "curso", new CourseDto(2L, "Bootstrap", 16)))
        .andExpect(content().string(containsString("Bootstrap")));
}

@Test
void erroresConservanEstadosHttp() throws Exception {
    mvc.perform(get("/cursos/999"))
        .andExpect(status().isNotFound());
    mvc.perform(get("/cursos").param("q", "x".repeat(61)))
        .andExpect(status().isBadRequest());
}

@Test
void portalRedirigeALaVistaDinamica() throws Exception {
    mvc.perform(get("/portal"))
        .andExpect(status().isFound())
        .andExpect(header().string("Location", "/cursos"));
}
}
```

En Spring Boot 4.1, WebMvcTest se importa desde `org.springframework.boot.webmvc.test.autoconfigure`. Las pruebas verifican el nombre de vista, atributos del Model, contenido renderizado, estados de error y redirección. No sustituyen las pruebas de teclado, diseño responsivo o recursos del navegador.

Ejecutar las pruebas y crear el JAR

```
./mvnw clean test
./mvnw clean package
java -jar target/techlab-web-0.0.1-SNAPSHOT.jar
```

Criterio de éxito. El resumen de Maven debe indicar cero fallos y cero errores. Luego pruebe el JAR empaquetado; una aplicación que solo funciona desde el IDE todavía no está verificada.

14. Matriz de pruebas manuales

Caso	Acción	Resultado esperado	Estado
M1	GET /cursos	Tres tarjetas y HTML sin expresiones Thymeleaf	200
M2	GET /cursos?q=Spring	Una tarjeta y campo con Spring	200
M3	GET /cursos?q=NoExiste	Estado vacío y cero resultados	200

Caso	Acción	Resultado esperado	Estado
M4	GET /cursos/2	Detalle de Bootstrap con 16 horas	200
M5	GET /cursos/999	Vista 4xx y ruta informada	404
M6	q con 61 caracteres	Solicitud rechazada	400
M7	GET /api/v1/cursos	JSON de tres elementos	200
M8	GET /portal	Redirección a /cursos	302

Pruebas responsivas y de accesibilidad

- Pruebe 320, 768 y 1280 píxeles; no debe existir desplazamiento horizontal.
- Recorra enlaces, campo y botones solo con Tab y Shift Tab; el foco debe ser visible.
- Active el enlace Saltar al contenido y confirme que evita la navegación repetida.
- Compruebe que label está asociado con q y que los encabezados mantienen un orden lógico.
- Aumente el zoom a 200 por ciento y confirme que el contenido sigue legible y operable.
- Desactive temporalmente el CDN: el contenido semántico debe conservarse aunque pierda estilos.

15. Diagnóstico de errores frecuentes

Síntoma	Causa probable	Corrección
TemplateInputException	Nombre o carpeta de plantilla incorrectos	Verifique templates/cursos/lista.html y el retorno cursos/lista
Aparece \${curso.titulo}	La página se abrió como archivo o no fue procesada	Acceda por http://localhost y revise el controlador
Error al evaluar propiedad	Nombre distinto al componente del record	Use id, titulo y horas exactamente
Fragmento no encontrado	Ruta o firma no coincide	Compare fragments/layout y head(titulo)
CSS con 404	URL relativa fija	Use th:href=@{/css/site.css} y ubique el archivo bajo static
Cambio no visible	Proceso antiguo o caché del navegador	Reinicie, recargue sin caché y confirme el proceso activo
404 devuelve 500	Se capturó o reemplazó mal la excepción	Mantenga ResponseStatusException con NOT_FOUND

Método de depuración. Lea primero la primera excepción causada por su código, confirme la ruta solicitada en Network, revise el nombre lógico de la vista y luego verifique los atributos del Model. Cambiar varias piezas a la vez oculta la causa.

16. Reto de transferencia

Implemente una vista GET /cursos/resumen sin eliminar ninguna función existente. Debe mostrar la cantidad de cursos y la suma de horas. La ruta literal /resumen debe coexistir con /{id}.

- Agregue totalHoras a CourseService mediante mapToInt y sum.
- Agregue un método resumen en CourseViewController con dos atributos de Model.

- Cree templates/cursos/resumen.html reutilizando head, cabecera y pie.
- Añada un enlace al resumen desde el catálogo y un texto mediante messages.properties.
- Pruebe nombre de vista, total de cursos y total de horas con MockMvc.
- Resultado esperado con los datos base: 3 cursos y 48 horas.

Restricción. No calcule el total dentro de la plantilla, no duplique la lista y no convierta el reto en una nueva API. La vista presenta el resultado de una operación de servicio.

17. Comprobación conceptual y cierre

1. Explique por qué devolver cursos/lista desde @Controller no escribe ese texto en la respuesta.
2. ¿Qué diferencia existe entre `${curso.titulo}` y `*{titulo}`?
3. ¿Por qué `@{}` es preferible a concatenar rutas manualmente?
4. ¿Qué problema de mantenimiento resuelve un fragmento?
5. ¿Por qué la búsqueda utiliza GET y conserva q en la URL?
6. ¿Qué riesgo introduce `th:utext` cuando recibe contenido no confiable?
7. ¿Qué demuestra MockMvc y qué debe verificarse todavía en un navegador?
8. ¿Por qué el origen de datos no debe consultarse desde una plantilla?

Cierre. TechLab ya ofrece dos representaciones sobre el mismo servicio: JSON para clientes y HTML para navegador. Esta separación prepara la semana 8, donde se consolidará Spring Web con Thymeleaf como segundo avance del proyecto formativo.

Referencias oficiales

- [Spring Boot 4.1.1 Web](#)
- [Spring Boot Servlet Web Applications](#)
- [Spring Boot Build Systems and Starters](#)
- [Thymeleaf 3.1.5 Tutorial Using Thymeleaf](#)
- [Thymeleaf 3.1.5 Integration with Spring](#)
- [Bootstrap 5.3.8 Getting Started](#)